TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

KĨ THUẬT

√CHIA CĂN

Từ cơ bản đến nâng cao

TẬP 1 · LÝ THUYẾT



Lê Thành Nam

01 Chia dãy thành khối

02 Ngưỡng căn

03 Thuật toán Mo

04 Chia căn nâng cao

12 bài học · 4 chương · hình vẽ từng bước, số đo thật

Dành cho học sinh THPT ôn luyện học sinh giỏi cấp tỉnh và tiếp cận cấp quốc gia

Tổng hợp và biên soạn: Lê Thành Nam

LỜI NÓI ĐẦU

Trong lập trình thi đấu có một con số xuất hiện ở khắp nơi một cách bí ẩn: √**n**. Chia dãy thành √n khối, mỗi khối √n phần tử. Đỉnh có hơn √m hàng xóm thì "nặng". Câu hỏi được sắp theo khối cỡ √n. Cứ √n thao tác thì dựng lại cấu trúc. Vì sao lại là căn bậc hai, chứ không phải một nửa, một phần mười hay log n?

Câu trả lời giản dị đến bất ngờ: hầu như mọi bài toán chia căn đều có **hai chi phí kéo ngược nhau**. Khối càng to thì số khối càng ít nhưng phần lẻ càng dài; ngưỡng càng cao thì đối tượng "lớn" càng hiếm nhưng mỗi đối tượng "nhỏ" càng tốn. Hai chi phí dạng n/B và B gặp nhau đúng tại B = √n. Hiểu được điều đó, em sẽ không còn **học thuộc** kĩ thuật chia căn, mà sẽ tự **thiết kế** được nó cho những bài toán chưa từng gặp.

Chia căn có một vị trí đặc biệt trong đề thi học sinh giỏi. Nó thường chậm hơn các cấu trúc dữ liệu "chuẩn" như cây phân đoạn một chút, nhưng **ngắn hơn, dễ đúng hơn**, và quan trọng hơn cả: nó giải được những bài mà cây phân đoạn bó tay. Số giá trị khác nhau trên đoạn, cộng đoạn rồi đếm số phần tử vượt ngưỡng, đếm tam giác trên đồ thị, tần số lớn nhất trên đoạn với câu hỏi trực tuyến... đều là những bài mà lời giải chia căn vừa tự nhiên vừa đủ nhanh.

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 12 bài học chia thành 4 chương: Chia dãy thành khối; Ngưỡng căn: nhiều cái nhỏ, ít cái lớn; Thuật toán Mo; Chia căn nâng cao.
- **Tập 2: Bài tập** gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ, có ràng buộc theo subtask như đề thi thật.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm, kể cả những cách **sai** hoặc **đúng nhưng chậm**.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, **đo thời gian chạy thật** để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Các con số trong sách (thời gian, số bước của con trỏ, đường cong theo kích thước khối) đều là số đo thật trên máy, không phải ước lượng. Mọi chương trình đều đã được đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường:

Phần	Nội dung
Khởi động	Một tình huống đời thường dẫn tới ý tưởng của bài học.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Cách ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm số đo thời gian thật.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: ý tưởng, hình vẽ, chạy tay, chứng minh độ phức tạp, chương trình.
Chọn kích thước khối	Phân tích vì sao $\sqrt{n}$ (hoặc $n^{2/3}$, $\sqrt{(n \log n)}$...), kèm số đo kiểm chứng.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
BÀI TOÁN	Đề bài chính thức.
Ý TƯỞNG CHIA CĂN	Cách chia, bản tóm tắt của khối, cách trả lời câu hỏi.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và số đo thời gian thật.
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn hoặc độ phức tạp.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Trong các hình vẽ, màu sắc có ý nghĩa cố định: ô **vàng** là **phần lẻ** (xử lí từng phần tử); ô **xanh lá** là **khối nằm gọn** (dùng bản tóm tắt) hoặc phần đang được tính; ô **xanh dương** là phần đã xử lí xong; ô **trắng** là phần nằm ngoài; ô **tím** là nhãn lười; ô **cam** là phần vừa bị sửa; ô **đỏ** là phần bị loại hoặc cách làm sai. Đường nét đứt màu xám bao quanh mỗi **khối**.

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một kĩ thuật chia căn, hãy tự trả lời ba câu hỏi: **chia cái gì** (vị trí, giá trị, đối tượng, câu hỏi hay thời gian), **hai chi phí nào kéo ngược nhau**, và **chúng gặp nhau ở đâu**. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **3**

CHƯƠNG 1. CHIA DÃY THÀNH KHỐI **5**

Bài 1. Chia dãy thành √n khối 5
Bài 2. Nhãn lười trên khối 15
Bài 3. Khối động: chèn, xoá và xây lại 23

CHƯƠNG 2. NGƯỠNG CĂN: NHIỀU CÁI NHỎ, ÍT CÁI LỚN **32**

Bài 4. Bước nhảy nhỏ, bước nhảy lớn 32
Bài 5. Đỉnh nặng, đỉnh nhẹ 40
Bài 6. Ít giá trị khác nhau 47

CHƯƠNG 3. THUẬT TOÁN MO **53**

Bài 7. Thuật toán Mo 53
Bài 8. Mo có cập nhật 61
Bài 9. Mo chỉ thêm (quay lui) 67

CHƯƠNG 4. CHIA CĂN NÂNG CAO **73**

Bài 10. Mo trên cây 73
Bài 11. Xử lí truy vấn theo lô 81
Bài 12. Tính trước theo cặp khối 88

PHỤ LỤC: BẢNG TRA CỨU NHANH **95**

Lê Thành Nam

CHƯƠNG 1

CHIA DÃY THÀNH KHỐI

*Muốn đếm số người trong một sân vận động, không ai đếm từng người một: ta đếm số người trong mỗi **khu khán đài**, ghi lại, rồi cộng các khu. Khi một người đổi chỗ, chỉ con số của hai khu thay đổi. Chương này biến ý tưởng giản dị ấy thành một kĩ thuật lập trình: chia dãy n phần tử thành khoảng √n **khối**, mỗi khối có khoảng √n phần tử, và lưu sẵn một "bản tóm tắt" cho từng khối. Mọi câu hỏi trên một đoạn khi đó chỉ chạm tới **không quá** √n **phần tử lẻ** và **không quá** √n **khối**. Em sẽ học cách tóm tắt một khối (tổng, max, dãy đã sắp xếp), cách ghi "nhãn lười" khi cả khối cùng bị sửa, và cách cho khối co giãn khi dãy bị chèn, xoá, đảo.*

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Chia dãy thành √n khối
- **Bài 2.** Nhãn lười trên khối
- **Bài 3.** Khối động: chèn, xoá và xây lại

Lê Thành Nam

BÀI 1 Chia dãy thành √n khối

Học xong bài này, em sẽ:

✓ Thấy rõ vì sao cả hai cách quen thuộc (duyệt thẳng, tổng tiền tố) đều **chậm** khi dãy vừa bị sửa vừa bị hỏi liên tục.

✓ Nắm được **ba vùng** của một đoạn hỏi: phần lẻ bên trái, các khối nằm gọn, phần lẻ bên phải.

✓ Cài đặt được bài **gán một phần tử, hỏi tổng một đoạn** với độ phức tạp O(1) mỗi lần gán và O(√n) mỗi lần hỏi.

✓ Hiểu vì sao kích thước khối tốt nhất là khoảng √**n**, kiểm chứng bằng số đo thật.

✓ Biết thay "tổng" bằng một bản tóm tắt khác: max, số cặp kề bằng nhau, hay cả một **dãy đã sắp xếp**.

■ KHỞI ĐỘNG

Một thư viện có 200 000 cuốn sách xếp thành một hàng dài, cuốn thứ i dày a_i milimét. Thủ thư liên tục thay sách (cuốn i được thay bằng cuốn khác độ dày v) và liên tục được hỏi: "Các cuốn từ số l tới số r chiếm bao nhiêu milimét giá sách?". Nếu mỗi câu hỏi em cầm thước đo từng cuốn, một câu hỏi có thể tốn 200 000 lần đo. Nếu em ghi sẵn trên một tờ giấy "tổng độ dày từ cuốn 1 tới cuốn i" cho mọi i, thì mỗi lần thay một cuốn sách lại phải sửa tới 200 000 con số. Có cách nào ở **giữa** hai thái cực ấy không?

Gợi ý từ đời thực: thư viện nào cũng chia sách thành **từng kệ**. Hãy ghi tổng độ dày của **mỗi kệ** lên nhãn của kệ đó...

■ BÀI TOÁN: KHOITONG

Cho dãy n số nguyên $a_1, a_2, ..., a_n$ và q thao tác thuộc một trong hai loại:

- **1 i v**: gán $a_i = v$.
- **2 l r**: in ra $a_l + a_{l+1} + ... + a_r$.

Dữ liệu vào: Dòng đầu gồm n, q ($1 \le n, q \le 2 \cdot 10^5$). Dòng thứ hai gồm n số a_i. Mỗi dòng trong q dòng sau là một thao tác. Mọi giá trị có trị tuyệt đối không quá 10^9. **Kết quả:** Với mỗi thao tác loại 2, in một dòng chứa đáp án.

Dữ liệu vào	Kết quả	Giải thích
16 3 3 1 4 1 5 9 2 6 5 3 5 8 9 7 9 3 2 3 14 1 6 0 2 3 14	64 55	$4 + 1 + 5 + 9 + 2 + 6 + 5 + 3 + 5 + 8 + 9 + 7 = 64$. Sau khi gán $a_6 = 0$ (trước đó $a_6 = 9$), tổng giảm 9 thành 55.

Lê Thành Nam

Đây là bài toán "cửa ngõ" của cả cuốn sách. Ta sẽ giải nó bằng ba cách, đo thời gian cả ba, và rút ra một khuôn mẫu dùng được cho hàng chục bài khác.

1. Hai cách quen thuộc và vì sao cả hai đều chậm

Cách 1: Duyệt thẳng

Lưu nguyên dãy. Gán: O(1). Hỏi: cộng từ a_l tới a_r, tốn $r - l + 1$ phép cộng.



Hình 1.1. Cách duyệt thẳng: câu hỏi tổng a[3..14] cộng lần lượt 12 phần tử

Cách 2: Tổng tiền tố

Lưu $P_i = a_1 + ... + a_i$. Hỏi: $P_r - P_{l-1}$, O(1). Nhưng gán $a_i = v$ làm thay đổi **mọi** P_j với $j \geq i$: tốn tới n phép tính.

■ ĐẾM SỐ PHÉP TÍNH

Trường hợp xấu nhất với $n = q = 2 \cdot 10^5$:

- **Duyệt thẳng:** mọi thao tác đều hỏi gần cả dãy: $q \cdot n = 4 \cdot 10^{10}$ phép cộng. Đo thật trên test 200 000 thao tác (đoạn hỏi dài gần n): **10,1 giây**.
- **Tổng tiền tố:** một nửa thao tác là gán ở đầu dãy, mỗi lần sửa gần n tổng tiền tố. Đo thật: **3,9 giây**.
- Giới hạn thời gian thường là 1 giây.

■ BẾ TẮC

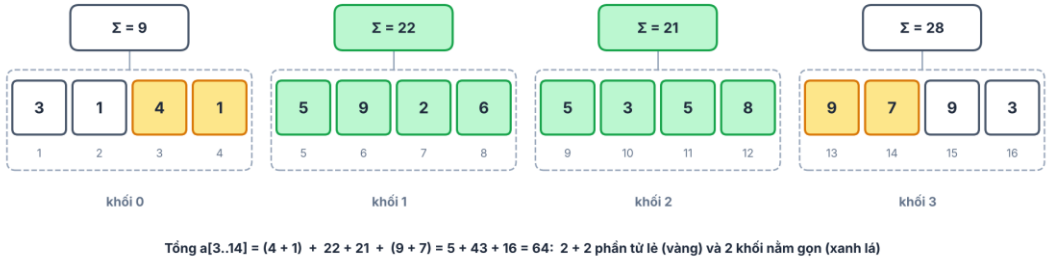
Cách 1 thì gán nhanh, hỏi chậm. Cách 2 thì hỏi nhanh, gán chậm. Lí do: cách 1 **không lưu gì**, cách 2 lưu **quá nhiều** (mỗi P_j phụ thuộc vào j phần tử, nên một phần tử ảnh hưởng tới n con số). Ta cần lưu "vừa phải": đủ để câu hỏi không phải cộng từng phần tử, nhưng mỗi phần tử chỉ ảnh hưởng tới **rất ít** con số đã lưu.

2. Phá bế tắc: Chia dãy thành khối

■ CHIA DÃY THÀNH KHỐI

- Chọn một số nguyên B (kích thước khối). Chia dãy thành các **khối** liên tiếp, mỗi khối B phần tử: khối 0 gồm các vị trí 0..B − 1, khối 1 gồm B..2B − 1, ... (đánh số từ 0). Khối cuối có thể ngắn hơn.
- Phần tử ở vị trí i (đếm từ 0) thuộc khối **i / B** (chia nguyên).
- Với mỗi khối b, lưu sẵn **blockSum[b]** = tổng các phần tử của khối đó.
- Mỗi phần tử chỉ thuộc **một** khối, nên chỉ ảnh hưởng tới **một** con số đã lưu.

Với 16 phần tử của ví dụ và B = 4, ta có 4 khối với tổng lần lượt là 9, 22, 21, 28.



Hình 1.2. *Dãy 16 phần tử chia thành 4 khối, mỗi khối ghi sẵn tổng. Câu hỏi [3, 14] chỉ cần 4 phần tử lẻ và 2 tổng khối*

Lê Thanh Nam

Trả lời câu hỏi: ba vùng

Đoạn [l, r] cắt dãy khối thành **ba vùng**:

- **Phần lẻ bên trái:** từ l tới hết khối chứa l. Cộng từng phần tử, không quá B phần tử.
- **Các khối nằm gọn:** mọi khối nằm hoàn toàn bên trong [l, r]. Cộng blockSum của chúng, không quá n/B khối.
- **Phần lẻ bên phải:** từ đầu khối chứa r tới r. Cộng từng phần tử, không quá B phần tử.

Nếu l và r **cùng một khối** thì không có khối nằm gọn nào, ta chỉ cần duyệt thẳng từ l tới r (không quá B phần tử).

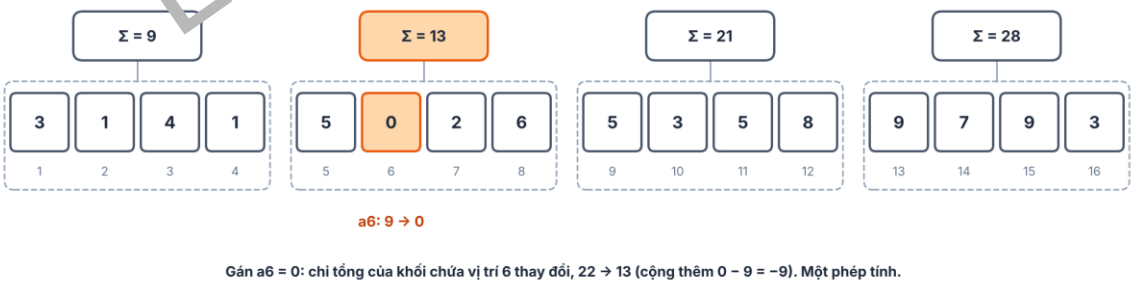
Chạy tay câu hỏi [3, 14] (đánh số từ 1, như trong đề):

Vùng	Các vị trí	Cộng gì	Kết quả
Lẻ bên trái	3, 4 (cuối khối 0)	$a_3 + a_4 = 4 + 1$	5
Khối nằm gọn	khối 1 (5..8), khối 2 (9..12)	blockSum[1] + blockSum[2] = 22 + 21	43
Lẻ bên phải	13, 14 (đầu khối 3)	$a_{13} + a_{14} = 9 + 7$	16
Tổng			**64**

Chỉ **6** phép cộng thay vì 12. Với dãy dài 200 000, khoảng cách còn lớn hơn rất nhiều: tối đa 2·447 + 447 ≈ 1 341 phép cộng thay vì 200 000.

Thao tác gán: sửa đúng một khối

Gán $a_i = v$ chỉ làm thay đổi tổng của **khối chứa i**. Không cần cộng lại cả khối: tổng mới = tổng cũ + $(v - a_i)$.



Hình 1.3. Gán $a_6 = 0$: chỉ tổng khối 1 thay đổi, 22 thành 13

Sau thao tác này, câu hỏi [3, 14] cho 5 + (13 + 21) + 16 = **55**, khớp với ví dụ.

Lê Thành Nam

3. Chương trình hoàn chỉnh

‹› Code ý chính: Câu hỏi tổng đoạn với ba vùng

```
long long s = 0;
int bl = l / B, br = r / B;                    // khối chứa l, khối chứa r
if (bl == br) {
    for (int i = l; i <= r; i++) s += a[i];    // cùng khối: duyệt thẳng
} else {
    for (int i = l; i < (bl + 1) * B; i++) s += a[i];    // lẻ trái
    for (int b = bl + 1; b < br; b++) s += blockSum[b];  // khối nằm gọn
    for (int i = br * B; i <= r; i++) s += a[i];         // lẻ phải
}
```

Hãy để ý ba vòng lặp: giới hạn của chúng được viết **bằng chỉ số khối**, không cần kiểm tra "phần tử này có thuộc khối nằm gọn không". Đầu khối b là b·B, cuối khối b là (b + 1)·B − 1.

▣ KHOITONG.CPP: Chia dãy thành khối, gán O(1), hỏi O(√n)

```
#include <bits/stdc++.h>
using namespace std;

int n, q, B;                    // B: kích thước một khối
vector<long long> a, blockSum;  // blockSum[b]: tổng các phần tử của khối b

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("KHOITONG.INP", "r")) {
        freopen("KHOITONG.INP", "r", stdin);
        freopen("KHOITONG.OUT", "w", stdout);
    }
    cin >> n >> q;
    B = max(1, (int)sqrt(n));
    a.assign(n, 0);
    blockSum.assign(n / B + 1, 0);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        blockSum[i / B] += a[i];          // phần tử i thuộc khối i / B
    }
    while (q--) {
        int type;
        cin >> type;
        if (type == 1) {                  // gán a[i] = v
            int i; long long v;
            cin >> i >> v; i--;
            blockSum[i / B] += v - a[i];  // sửa tổng của đúng một khối
            a[i] = v;
        } else {                          // tổng a[l..r]
            int l, r;
            cin >> l >> r; l--; r--;
            long long s = 0;
            int bl = l / B, br = r / B;
            if (bl == br) {               // l, r cùng một khối: duyệt thẳng
                for (int i = l; i <= r; i++) s += a[i];
            } else {
                for (int i = l; i < (bl + 1) * B; i++) s += a[i];   // lẻ trái
                for (int b = bl + 1; b < br; b++) s += blockSum[b];  // khối gọn
                for (int i = br * B; i <= r; i++) s += a[i];         // lẻ phải
            }
            cout << s << '\n';
        }
    }
```

Lê Thành Nam

```
45      return 0;
46  }
```

4. Chọn kích thước khối

Một câu hỏi tốn tối đa khoảng **2B + n/B** bước: hai phần lẻ, mỗi phần dưới B phần tử, cộng với n/B khối nằm gọn.

- B quá **nhỏ** (ví dụ B = 1): mỗi khối chỉ có một phần tử, n/B = n khối. Ta quay về cách duyệt thẳng.
- B quá **lớn** (ví dụ B = n): chỉ có một khối, phần lẻ dài tới n. Cũng quay về cách duyệt thẳng.
- Ở giữa có một điểm cân bằng.



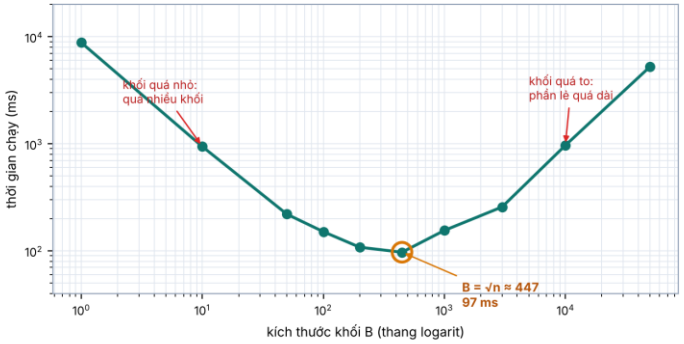
Hình 1.4. Ba kích thước khối cho cùng một câu hỏi: khối nhỏ thì nhiều khối, khối to thì phần lẻ dài

■ VÌ SAO B ≈ √n LÀ TỐT NHẤT?

Hai thành phần 2B và n/B "kéo" theo hai hướng ngược nhau: B tăng thì 2B tăng, n/B giảm. Tổng hai đại lượng như vậy nhỏ nhất (cỡ) khi chúng **xấp xỉ bằng nhau**: B ≈ n/B, tức $B^2 \approx n$, **B ≈ √n**. Khi đó một câu hỏi tốn khoảng 3√n bước.

Chính xác hơn, theo bất đẳng thức Cô-si: $2B + n/B \geq 2 \cdot \sqrt{(2B \cdot n/B)} = 2\sqrt{(2n)}$, dấu bằng khi $B = \sqrt{(n/2)}$. Hằng số này không quan trọng: chọn B = √n hay √(n/2) chỉ chênh nhau vài phần trăm, như số đo bên dưới cho thấy.

Lí thuyết là vậy, còn thực tế thì sao? Ta chạy cùng chương trình với $n = q = 2 \cdot 10^5$ (đoạn hỏi dài gần n) và chỉ thay đổi B:



B	ms
1	8 828
10	943
50	220
100	151
200	108
447	**97**
1 000	155
3 000	257
10 000	967
50 000	5 253

Hình 1.5. Thời gian chạy thật theo kích thước khối: đường cong hình chữ U, đáy ở quanh $B = \sqrt{n} \approx 447$

Lê Thành Nam

■ ĐẾM SỐ PHÉP TÍNH

- B = 1 hoặc B = 50 000: **5 đến 9 giây**, không khác gì duyệt thẳng.
- B từ 200 tới 1 000: **0,10 đến 0,16 giây**. Đáy đường cong khá **bằng phẳng**: chọn B lệch √n gấp đôi cũng chỉ chậm thêm vài chục phần trăm.
- B = √n ≈ 447: **97 ms**. So với duyệt thẳng (10,1 giây) nhanh hơn khoảng **100 lần**; so với tổng tiền tố (3,9 giây) nhanh hơn khoảng **40 lần**.
- Độ phức tạp cả bài: O(n) khởi tạo, O(1) mỗi lần gán, O(√n) mỗi lần hỏi: **O(n + q√n)**. Với n = q = $2 \cdot 10^5$: khoảng $2 \cdot 10^5 \cdot 3 \cdot 447 \approx 2{,}7 \cdot 10^8$ phép cộng rất đơn giản.

■ LƯU Ý

Đáy bằng phẳng của đường cong là một tin vui: em không cần chọn B thật chính xác. Nhưng khi hai loại thao tác có chi phí **khác nhau** (ví dụ gán tốn O(B) còn hỏi tốn O(n/B · log B)), điểm cân bằng sẽ lệch khỏi √n. Ta sẽ gặp điều này ở bài DEMNHO cuối bài học và ở Bài 2.

5. Khuôn mẫu chung: "tóm tắt" một khối

Toàn bộ lời giải chỉ dựa vào hai câu hỏi:

■ HAI CÂU HỎI THIẾT KẾ CHIA KHỐI

1. **Mỗi khối lưu gì** (bản tóm tắt) để một khối nằm gọn được xử lí **nhanh** mà không duyệt từng phần tử?
2. **Khi một phần tử thay đổi**, bản tóm tắt của khối chứa nó được sửa **thế nào**, tốn bao lâu?

Phần lẻ thì luôn duyệt từng phần tử, không cần nghĩ.

Lê Thành Nam

Thay "tổng" bằng đại lượng khác, ta được một loạt bài toán:

Câu hỏi trên đoạn	Mỗi khối lưu	Xử lí khối nằm gọn	Sửa khi gán một phần tử
Tổng	tổng khối	cộng tổng khối, O(1)	cộng thêm v − a_i, O(1)
Max (gán tùy ý)	max khối	so sánh, O(1)	**tính lại** max cả khối, O(B)
Số i có $a_i = a_{i+1}$	số cặp kề bằng nhau trong khối	cộng, O(1)	chỉ hai cặp (i − 1, i), (i, i + 1) thay đổi, O(1)
Số phần tử ≤ x	**dãy đã sắp xếp** của khối	tìm kiếm nhị phân, O(log B)	xoá giá trị cũ, chèn giá trị mới, O(B)
Tần số của giá trị x	bảng đếm của khối	tra bảng, O(1)	giảm đếm cũ, tăng đếm mới, O(1)

Hãy để ý dòng thứ hai: với **max**, khi gán a_i = v nhỏ hơn giá trị cũ, max của khối có thể giảm, và ta **không biết** giảm xuống bao nhiêu nếu không xem lại cả khối. Đó là lí do max phải tính lại trong

O(B). Vẫn ổn: gán tốn O(√n), hỏi tốn O(√n). Còn dòng thứ tư cho thấy bản tóm tắt có thể là một cấu trúc lớn, không chỉ một con số.

6. So sánh với cây phân đoạn

Nếu em đã biết cây phân đoạn (segment tree), em sẽ thấy nó giải bài KHOITONG trong O(log n) mỗi thao tác, nhanh hơn O(√n). Vậy học chia khối để làm gì?

Tiêu chí	Chia khối	Cây phân đoạn
Độ phức tạp mỗi thao tác	O(√n)	O(log n)
Độ dài code	ngắn, khó sai	dài hơn, dễ sai chỉ số
Bản tóm tắt phải **ghép được** từ hai nửa?	**Không cần**	Bắt buộc
Bản tóm tắt là dãy sắp xếp, bảng đếm...	tự nhiên	tốn O(n log n) bộ nhớ hoặc rất phức tạp
Dãy bị chèn, xoá, cắt dán	dễ (Bài 3)	cần cây cân bằng (treap)

Điểm then chốt là dòng thứ ba: cây phân đoạn chỉ dùng được khi thông tin của một đoạn **ghép được** từ hai nửa. Chia khối thì không cần điều đó: ở mỗi khối ta chỉ cần trả lời câu hỏi cho **chính khối đó**, rồi gộp các câu trả lời lại (thường là cộng). Nhiều bài trong cuốn sách này thuộc loại cây phân đoạn "không làm được" hoặc "làm được nhưng rất khó".

7. Mở rộng

Mở rộng 1: Khối có kích thước cố định

Nhiều người viết B là một hằng số, ví dụ `const int B = 450;`, thay vì tính √n. Như đường cong ở Hình 1.5 cho thấy, với n từ 10^5 tới $5 \cdot 10^5$ điều này hầu như không ảnh hưởng tới tốc độ, lại cho phép khai báo mảng tĩnh `blockSum[N / B + 1]`. Chỉ cần đảm bảo $B \geq 1$ và số khối đủ chỗ.

Mở rộng 2: Bảng hai chiều

Ý tưởng chia khối dùng được cả trên bảng n × n: chia bảng thành các ô vuông cạnh √n, mỗi ô lưu tổng. Câu hỏi hình chữ nhật gồm các ô vuông nằm gọn và các "viền" lẻ. Tuy nhiên với bảng, cây Fenwick hai chiều thường gọn hơn.

Mở rộng 3: Chia căn ở khắp nơi

"Chia thành √n phần, mỗi phần √n" là một ý tưởng xuất hiện ở rất nhiều nơi khác, không chỉ trên dãy. Chương 2 chia **các giá trị** thành nhỏ và lớn, Chương 3 chia **các câu hỏi** theo khối, Chương 4 chia **thời gian** thành từng lô. Tất cả đều dựa trên cùng một phép cân bằng: hai chi phí kéo ngược nhau, gặp nhau ở √n.

Lê Thành Nam

■ BẪY THƯỜNG GẶP

- **Lẫn lộn đánh số từ 0 và từ 1.** Công thức "phần tử i thuộc khối i / B" chỉ đúng khi đánh số từ 0. Đọc l, r rồi trừ 1 ngay.
- **Quên trường hợp l, r cùng khối.** Nếu viết thẳng ba vòng lặp, vòng "lẻ trái" chạy tới hết khối, vượt quá r, cộng thừa.
- **B = 0** khi n nhỏ (ví dụ (int)sqrt(0)). Luôn lấy B = max(1, (int)sqrt(n)).
- **Mảng blockSum thiếu chỗ:** số khối là [n/B], có thể lớn hơn n/B. Khai báo n/B + 1.
- **Tràn số:** tổng tới $2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}$, dùng `long long`.
- **Gán mà quên sửa tổng khối,** hoặc sửa tổng khối bằng v thay vì v − a_i.

■ GHI NHỚ

- Chia dãy thành các khối B ≈ √n phần tử; phần tử i (từ 0) thuộc khối i / B.
- Mỗi câu hỏi đoạn = **lẻ trái** (≤ B phần tử) + **khối nằm gọn** (≤ n/B khối) + **lẻ phải** (≤ B phần tử).
- Mỗi khối lưu một **bản tóm tắt**: tổng, max, số cặp, dãy sắp xếp... miễn là khối nằm gọn được xử lí nhanh và bản tóm tắt sửa được khi một phần tử đổi.
- Không cần ghép được hai nửa như cây phân đoạn: đây là sức mạnh lớn nhất của chia khối.

Lê Thành Nam

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	CAO NHẤT	Max trên đoạn, có gán. Mỗi khối lưu max; khi gán phải tính lại cả khối.
1.2	CẶP KỀ	Đếm i có $a_i = a_{i+1}$ trên đoạn. Mỗi khối đếm số cặp; một lần gán chỉ đổi hai cặp.
1.3	ĐẾM NHỎ	Đếm phần tử ≤ x trên đoạn, có gán. Mỗi khối lưu dãy đã sắp xếp.

Tự kiểm tra nhanh:

1. Với ví dụ ở Hình 1.2 (B = 4), câu hỏi [2, 15] gồm những vùng nào và kết quả bằng bao nhiêu? *(Đáp số: lẻ trái 1 + 4 + 1 = 6, khối 1 và 2 là 43, lẻ phải 9 + 7 + 9 = 25; tổng 74.)*

2. n = 10^6. Một câu hỏi theo cách chia khối tốn khoảng bao nhiêu bước? *(Đáp số: B = 1000, khoảng 2·1000 + 1000 = 3000 bước.)*

3. Nếu thao tác gán xảy ra 10^5 lần nhưng chỉ có 10 câu hỏi, em có nên dùng chia khối không? *(Đáp số: không cần; duyệt thẳng tốn 10 · n phép, đã đủ nhanh. Chia khối có lợi khi cả hai loại thao tác đều nhiều.)*

BÀI 2 Nhãn lười trên khối

Học xong bài này, em sẽ:

✓ Hiểu vì sao thao tác **sửa cả một đoạn** làm hỏng lời giải của Bài 1, và cách **nhãn lười** (lazy) cứu nó.

✓ Phân biệt rõ **giá trị riêng** lưu trong mảng và **giá trị thật** = giá trị riêng + nhãn của khối.

✓ Cài đặt bài **cộng đoạn, hỏi tổng đoạn** trong O(√n) mỗi thao tác.

✓ Biết xử lí nhãn **gán** (không cộng dồn được): phải **đẩy nhãn xuống** trước khi sửa lẻ.

✓ Biết kết hợp nhãn lười với khối đã sắp xếp, và chọn lại kích thước khối khi hai loại thao tác có chi phí lệch nhau.

■ KHỞI ĐỘNG

Một khu vườn có 200 000 cây trồng thành hàng. Mỗi buổi sáng, người làm vườn tưới thêm cho **cả một đoạn** cây từ cây l đến cây r, mỗi cây lớn thêm v milimet. Thỉnh thoảng chủ vườn hỏi: "Tổng chiều cao các cây từ l đến r là bao nhiêu?". Với cách chia khối của Bài 1, một lần tưới đoạn dài phải sửa từng cây một: 200 000 lần sửa. Nhưng hãy nghĩ như người làm vườn: khi tưới cho cả một luống, ta đâu cần ghi vào sổ chiều cao mới của từng cây. Chỉ cần ghi một dòng: "**cả luống này** đã được tưới thêm v".

■ BÀI TOÁN: CONGDOAN

Cho dãy n số nguyên $a_1, ..., a_n$ và q thao tác thuộc một trong hai loại:

- **1 l r v**: cộng v vào mọi a_i với $l \le i \le r$.
- **2 l r**: in ra $a_l + a_{l+1} + ... + a_r$.

Dữ liệu vào: Dòng đầu gồm n, q ($1 \le n, q \le 2 \cdot 10^5$). Dòng thứ hai gồm n số a_i ($|a_i| \le 10^9$). Mỗi dòng trong q dòng sau là một thao tác ($|v| \le 10^9$). **Kết quả:** Với mỗi thao tác loại 2, in một dòng chứa đáp án.

Dữ liệu vào	Kết quả	Giải thích
16 4 3 1 4 1 5 9 2 6 5 3 5 8 9 7 9 3 2 3 14 1 3 14 2 2 3 14 2 7 7	64 88 4	Ban đầu tổng là 64 (như Bài 1). Cộng 2 vào 12 phần tử: tổng tăng 24 thành 88. Phần tử $a_7 = 2$ trở thành 4.

Lê Thành Nam